Belajar_Teleportasi

April 15, 2025

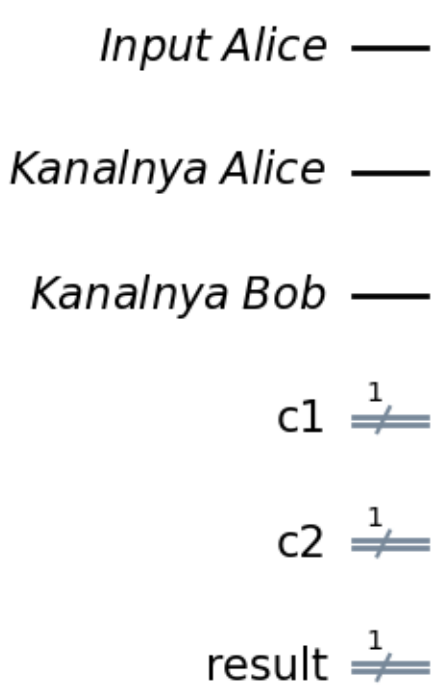
[ ]:
```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit.circuit.library import UGate
from numpy import pi
```

[ ]:
```
Input = QuantumRegister(1, "Input Alice")
Alice = QuantumRegister(1, "Kanalnya Alice")
Bob = QuantumRegister(1, "Kanalnya Bob")
c1 = ClassicalRegister(1, "c1")
c2 = ClassicalRegister(1, "c2")
Result = ClassicalRegister(1, "result")
```

[ ]:
```
qc = QuantumCircuit(Input, Alice, Bob, c1, c2, Result)
```

[ ]:
```
qc.draw('mpl')
```

[ ]:

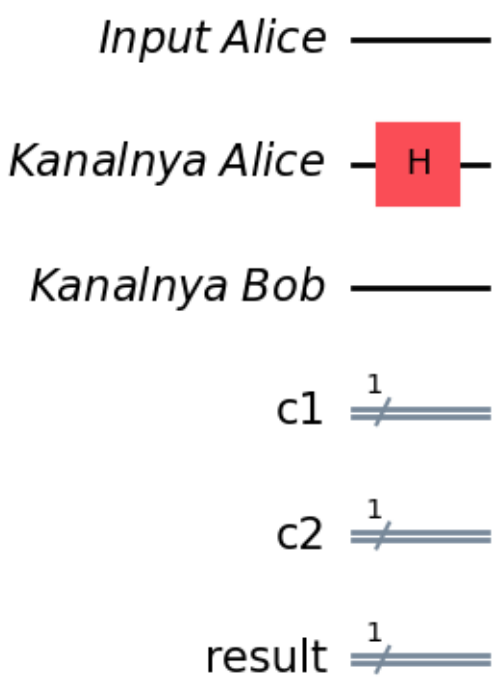


[ ]:
```
qc.h(Alice)
```

[ ]: <qiskit.circuit.instructionset.InstructionSet at 0x7e8988570e50>

[ ]:
```
qc.draw('mpl')
```

[ ]:

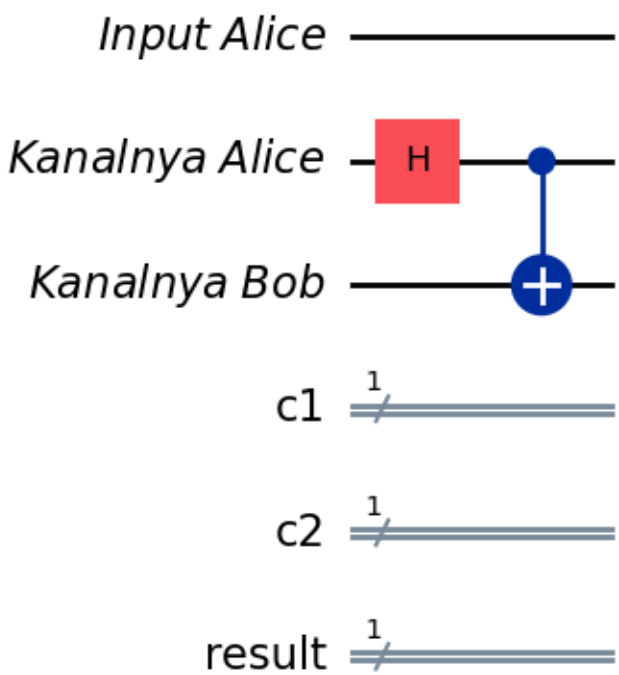


[ ]:
```
qc.cx(Alice,Bob)
```

[ ]: <qiskit.circuit.instructionset.InstructionSet at 0x7e8988571db0>

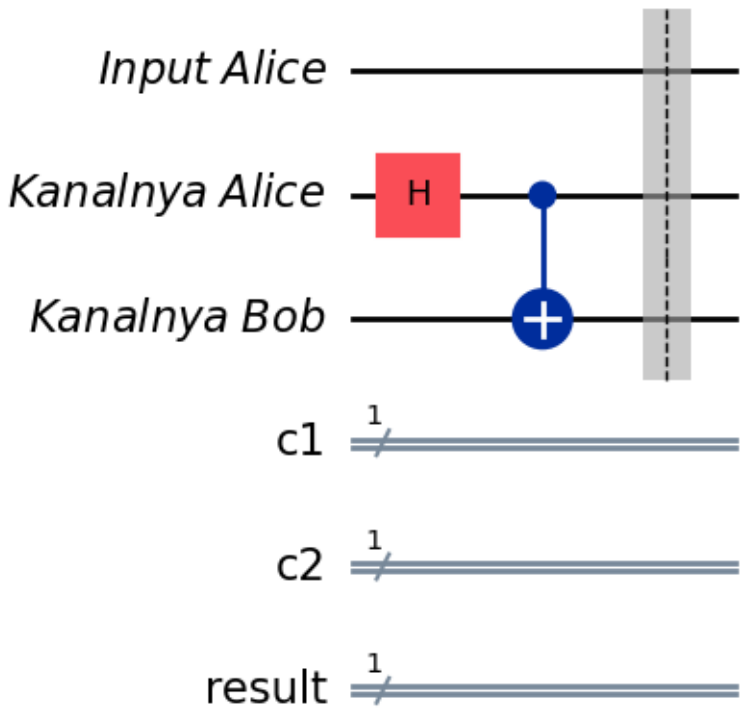
[ ]:
```
qc.draw('mpl')
```

[ ]:

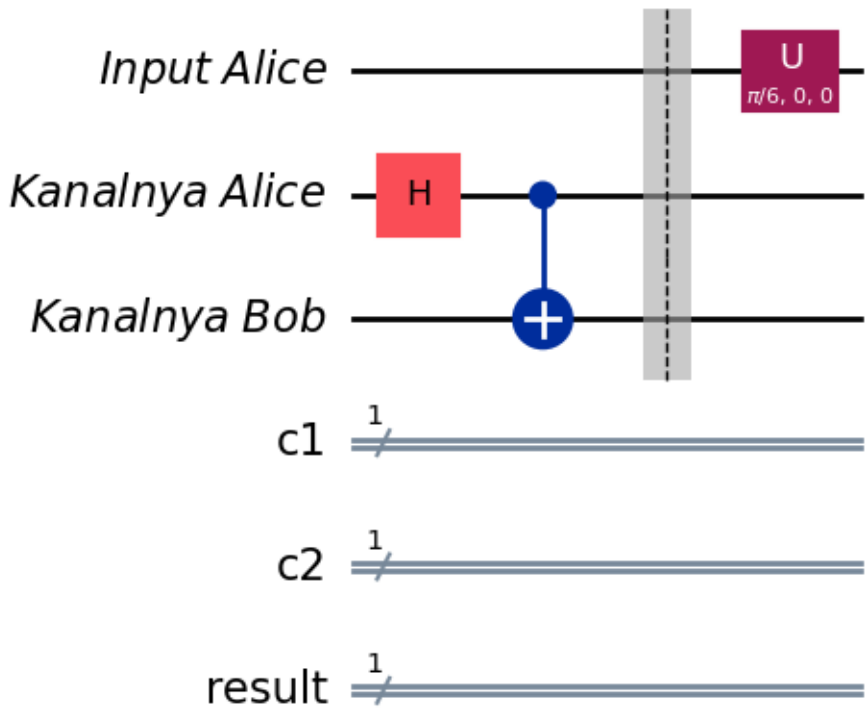


[ ]:
```
qc.barrier()
qc.draw('mpl')
```

[ ]:

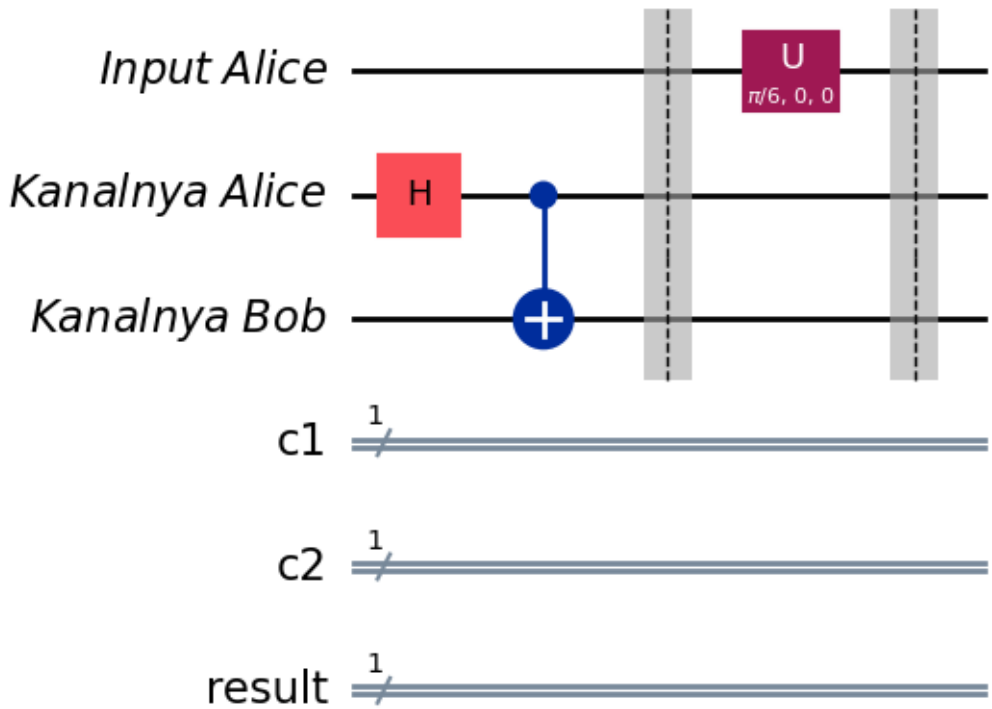



```
[ ]: Gate_Input = UGate(theta = 30 * 2 * pi / 360, phi = 0 * 2 * pi / 360, lam = 0 *␣
     ↪2 * pi / 360)
     qc.append(Gate_Input,Input)
     qc.draw('mpl')
[ ]:
```

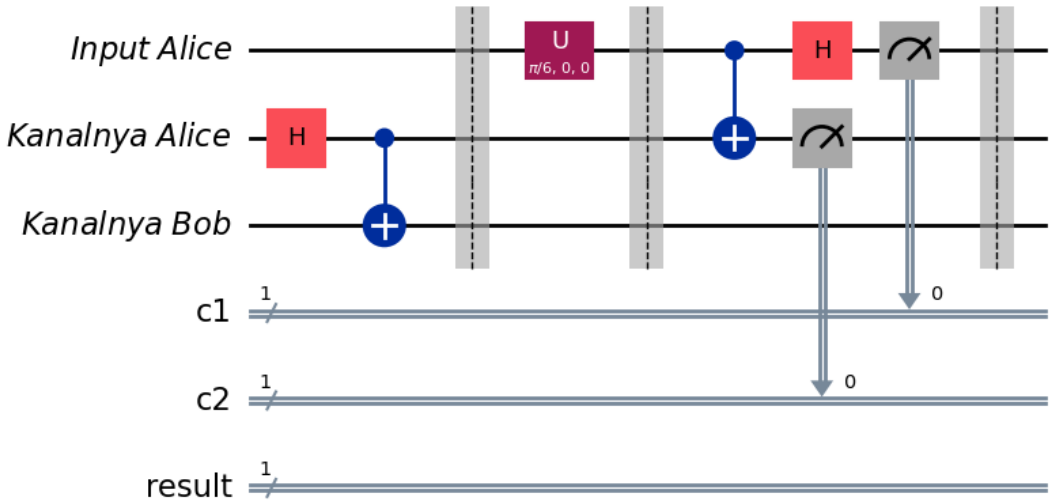


[ ]:
```
qc.barrier()
qc.draw('mpl')
```

[ ]:

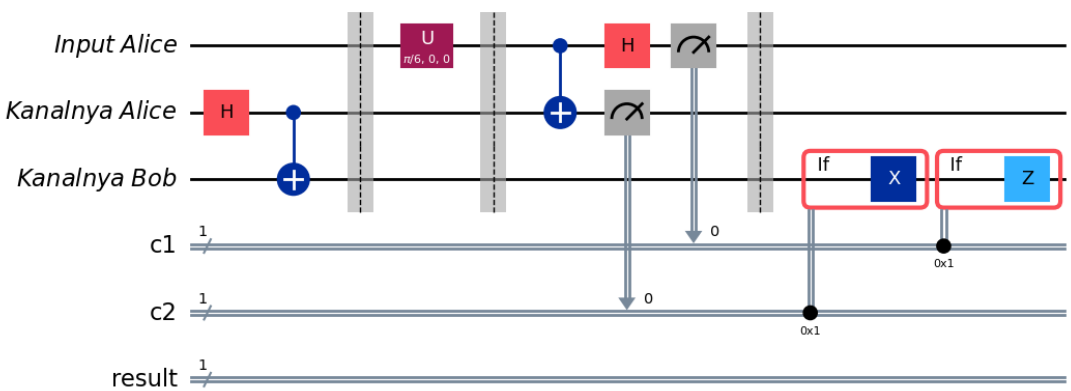



```
[ ]: qc.cx(Input,Alice)
     qc.h(Input)
     qc.measure(Input,c1)
     qc.measure(Alice,c2)
     qc.barrier()
     qc.draw('mpl')
[ ]:
```



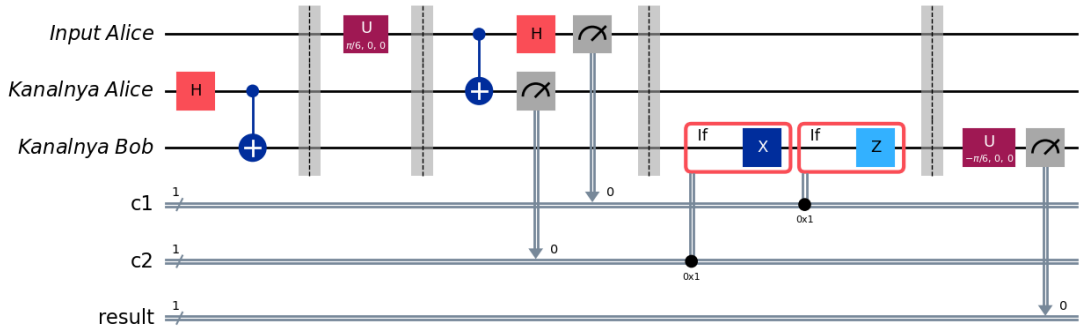

```
[ ]: with qc.if_test((c2,1)):
       qc.x(Bob)
     with qc.if_test((c1,1)):
       qc.z(Bob)
     qc.draw('mpl')
```

[ ]:




```
[ ]: qc.barrier()
     qc.append(Gate_Input.inverse(),Bob)
     qc.measure(Bob,Result)
     qc.draw('mpl')
```

[ ]:



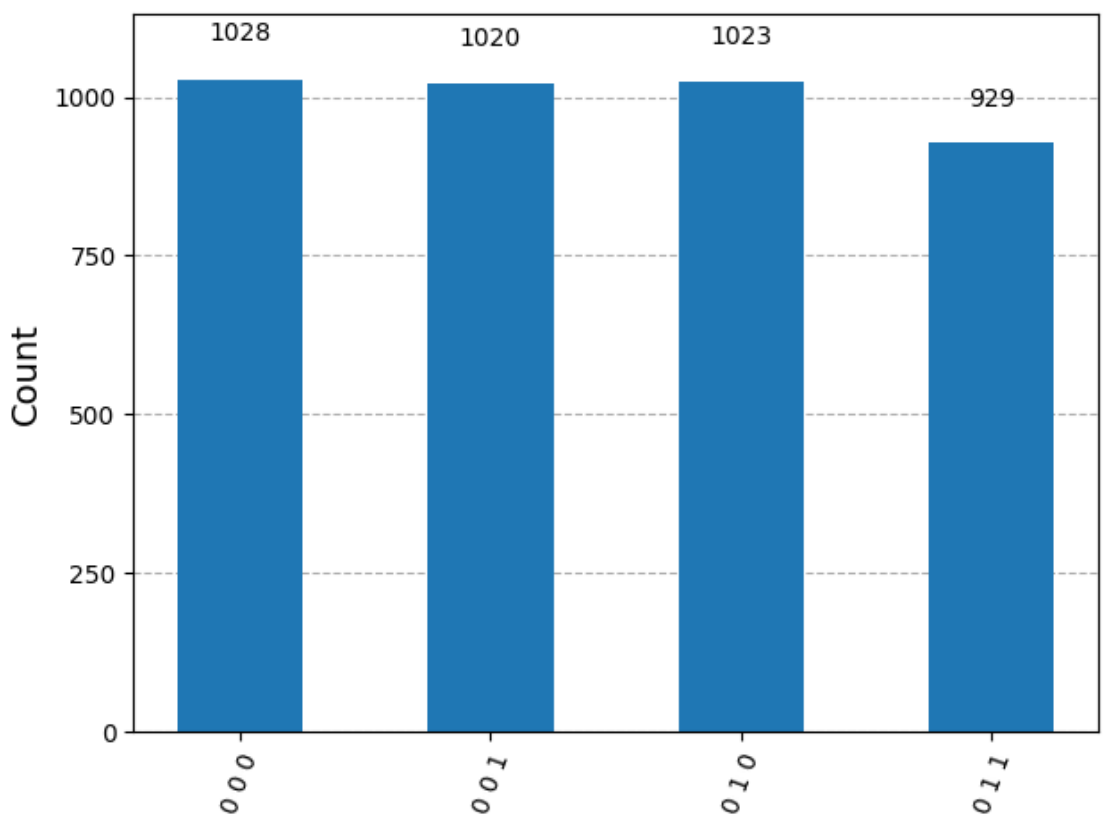

```
[ ]: from qiskit_aer import AerSimulator
```

```
[ ]: hasil_simulasi = AerSimulator(shots=4000).run(qc).result()
```

```
[ ]: statistik = hasil_simulasi.get_counts()
```

```
[ ]: from qiskit.visualization import plot_histogram
```

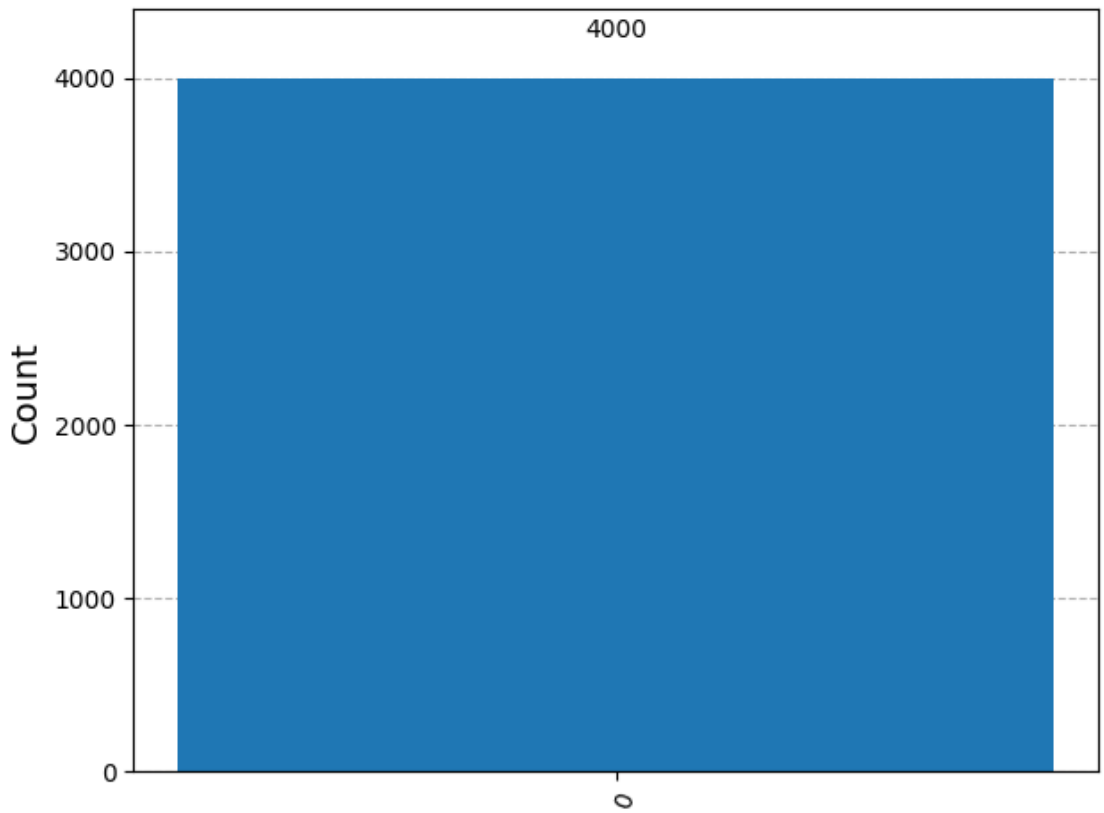
```
[ ]: display(plot_histogram(statistik))
```



[ ]:
```
from qiskit.result import marginal_distribution
```

[ ]:
```
filter_statistik = marginal_distribution(statistik, [2])
```

[ ]:
```
display(plot_histogram(filter_statistik))
```



[ ]:
```
from qiskit_ibm_runtime import QiskitRuntimeService
```

[ ]:
```
# Load saved credentials
service = QiskitRuntimeService()
```

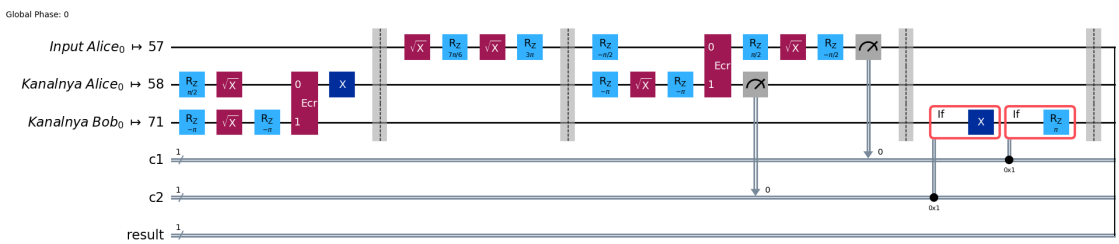
[ ]:
```
service.backends()
```

[ ]:
```
[<IBMBackend('ibm_brisbane')>,
 <IBMBackend('ibm_kyiv')>,
 <IBMBackend('ibm_sherbrooke')>]
```

[ ]:
```
from qiskit import transpile

backendIBM = service.backend("ibm_brisbane") #memilih backend

transpiled_circuit = transpile(qc, backendIBM) #menyesuaikan dan␣
↪mengoptimalisasi ke sistem sesuai backend yang diambil
```

[ ]:
```
transpiled_circuit.draw('mpl')
```

[ ]:



[ ]:
```
from qiskit_ibm_runtime import SamplerV2 as Sampler
```

[ ]:
```
sampler = Sampler(backendIBM)
job = sampler.run([transpiled_circuit])
print(f"job id: {job.job_id()}")
```

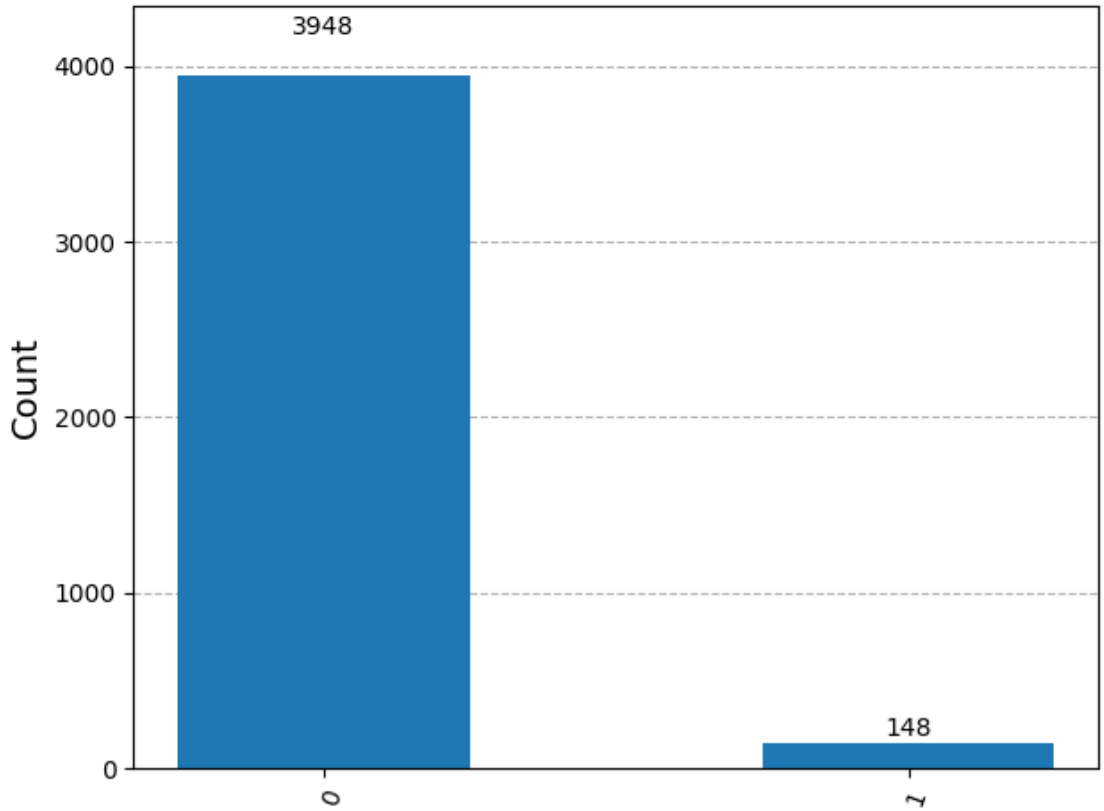
```
job id: czyxaktd8drg008hz1zg
```

[ ]:
```
from qiskit_ibm_runtime import QiskitRuntimeService

service = QiskitRuntimeService(
    channel='ibm_quantum',
    instance='ibm-q/open/main',
   ␣
 ↪token='a6bf19c35e6726dbf4c0ec5048befd6a1bd99664513cddf865f9558ed32e44c728ca15ee2115be167a00
)
job = service.job('czyxaktd8drg008hz1zg')
job_result = job.result()

# To get counts for a particular pub result, use
#
```

```
# pub_result = job_result[<idx>].data.<classical register>.get_counts()
#
# where <idx> is the index of the pub and <classical register> is the name of␣
↪the classical register.
# You can use circuit.cregs to find the name of the classical registers.
```

[ ]:
```
pub_result = job_result[0].data.result.get_counts()
```

[ ]:
```
display(plot_histogram(pub_result))
```



[ ]:
```

```